

Learning with Rare Success but Rich Feedback via Reflection-Enhanced Self-Distillation

Yuwei Zhang^{1*} Sha Li² Changlong Yu² Qin Lu² Shuwei Jin² Chengyu Dong²
Haoran Liu¹ Ilgee Hong^{3*} Xintong Li^{1*} Zhenyu Shi² Bing Yin² Jingbo Shang^{1†}
UC San Diego¹ Amazon² Georgia Institute of Technology³
{yuz163, jshang}@ucsd.edu

通过反思增强的自蒸馏，在成功稀少但反馈丰富的条件下学习

[Project Page](#) [GitHub](#)

Abstract

Enabling Large Language Models (LLMs) to continuously improve from environmental interactions is a central challenge in post-training. 让大语言模型 (LLM) 能够通过环境交互持续改进，是后训练阶段的一个核心挑战。While on-policy self-distillation offers a promising paradigm, existing methods predominantly treat environmental feedback as a passive conditioning signal. 尽管on-policy 自蒸馏提供了一条很有前景的路线，但现有方法大多把环境反馈仅仅当作被动的条件信号。Consequently, they heavily rely on successful demonstrations and struggle to learn in rare-success regimes. 因此，它们高度依赖成功示范，在成功样本极其稀少的场景中往往很难学起来。^{[注]1}

[重点] To bridge this gap, we introduce Reflection-Enhanced Self-Distillation (RESL), a framework that transforms raw failure feedback into an active source of corrective supervision. 为了解决这个问题，我们提出Reflection-Enhanced Self-Distillation (RESL)，把原始失败反馈转化为主动的纠错监督来源。Instead of passively appending feedback, RESL interprets failed trajectories by generating retrospective reflections to diagnose local errors, and curates a persistent global playbook to preserve reusable lessons across training steps. 与其被动拼接反馈，RESL 会先为失败轨迹生成回溯式反思，用来定位局部错误；再维护一个持久化的全局playbook，把可复用经验保留下来，跨训练步复用。The enriched context enables the self-teacher to provide actionable token-level supervision even in the absence of successful rollouts. 这套增强后的上下文，使self-teacher 即便在没有成功rollout 的情况下，也能提供可执行的token 级监督。^{[注]2}

Empirical evaluations on multiple continual learning tasks demonstrate that RESL substantially outperforms standard self-distillation baselines. 在多个持续学习任务上的实验表明，RESL 明显优于标准自蒸馏基线。**[重点]** Furthermore, RESL achieves significantly faster early-stage improvement than GRPO with 8× samples using only a single rollout per prompt, highlighting its superior interaction efficiency. 此外，RESL 只用每个prompt 一次rollout，就能在训练早期比使用8× 样本的GRPO 提升得更快，凸显了它在交互效率上的优势。^{[注]3}

*Work done during internship.

†Corresponding authors.

¹【空缺|保留】

评：摘要开头很快点明了当前自蒸馏在稀有成功场景里的真正瓶颈，问题定义清楚而且聚焦。

²【方法|保留】

评：方法段不仅说了提出了什么，还交代了reflection 和playbook 各自负责什么，因此摘要层面的机制解释已经足够具体。

³【结果|保留】

评：结果句同时给出了主要对比对象、早期阶段这一关键场景，以及单次rollout 的设置，摘要收束是成立的。

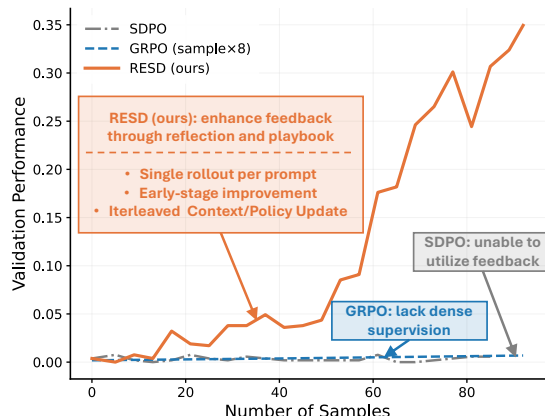


Figure 1: RESD improves interaction efficiency during training. The x-axis is the number of samples.

1 Introduction

引言

A fundamental challenge in the post-training of Large Language Models (LLMs) is enabling continuous improvement through environmental interactions. 大语言模型 (LLM) 后训练中的一个根本挑战，是如何让模型能够通过环境交互持续改进。Traditionally, Reinforcement Learning (RL) algorithms, such as PPO or GRPO, have been the standard paradigm for aligning models with desired outcomes. 传统上，PPO、GRPO 等强化学习 (RL) 算法一直是把模型对齐到目标结果上的标准范式。However, a critical bottleneck in deploying RL for complex, multi-step tasks is the reliance on sparse reward signals. 但在把RL 用到复杂的多步任务时，一个关键瓶颈在于它依赖稀疏奖励信号。In scenarios where successful trajectories are exceedingly rare, standard RL often struggles to effectively bootstrap policies because it lacks dense supervision to guide the model through a vast exploration space. 当成功轨迹极为罕见时，标准RL 往往难以有效启动策略学习，因为它缺少足够稠密的监督来引导模型穿过庞大的探索空间。[注]⁴

To mitigate the sparse reward problem, On-Policy Distillation (OPD) has emerged as a promising alternative [1, 12, 23]. 为了缓解稀疏奖励问题，On-Policy Distillation (OPD) 近来成为一条很有前景的替代路线 [1, 12, 23]。Rather than relying on a single scalar reward at the end of a rollout, OPD leverages a stronger or privileged teacher model to compute target probabilities for every single token in the generated trajectory. 它不再只依赖rollout 末尾的单个标量奖励，而是借助一个更强或具备特权信息的teacher，为生成轨迹中的每个token 计算目标概率。This mechanism elegantly translates sparse trajectory-level outcomes into dense, token-level learning signals. 这种机制把稀疏的轨迹级结果转化成了稠密的token 级学习信号。However, standard OPD requires maintaining a separate expert model, which incurs high computational costs and risks distribution mismatch between the teacher and the student. 但标准OPD 需要维护一个独立的专家模型，这既带来较高的计算成本，也容易让teacher 和student 之间出现分布失配。To address this, self-distillation variants like SDPO instantiate the teacher from the model itself [6, 30]. 为了解决这个问题，SDPO 这类自蒸馏方法会直接从模型自身实例化teacher [6, 30]。By conditioning the self-teacher on environmental feedback, SDPO eliminates the need for an external oracle while perfectly aligning the supervisory signal with the model's own generation distribution. 通过让self-teacher 以环境反馈为条件，SDPO 一方面不再需要外部oracle，另一方面也让监督信号与模型自身的生成分布保持一致。[注]⁵

While self-distillation resolves both reward sparsity and distribution mismatch, its success hinges entirely on the quality of the self-teacher's supervision. 虽然自蒸馏同时缓解了奖励稀疏和分布失配，但它能否奏效，最终仍取决于self-teacher 给出的监督质量。Current methods predominantly treat environmental feedback as a static, passive conditioning variable. 现有方法大多把环境反馈视为静态、被动的条件变量。[重点] We argue that this overlooks a key design choice: the structural representation of the feedback itself. 我们认为，这忽略了一个关键设计维度：反馈本身该如何被

⁴ 【背景|保留】

评：开头先把RL 的通用地位交代出来，再顺势收束到稀疏奖励这个真正障碍，问题提出比较自然。

⁵ 【文献定位|保留】

评：这一段把OPD 与self-distillation 的关系、各自优势和遗留问题交代得很清楚，为后文继续压缩到反馈表征问题做好了铺垫。

结构化表示。In sparse-reward regimes where successful demonstrations are absent, the teacher is forced to derive supervision almost entirely from failed rollouts. 在缺乏成功示范的稀疏奖励场景中, teacher 几乎只能从失败rollout 中提炼监督信号。We demonstrate in Figure 1 that simply feeding raw failure outcomes or unstructured environment errors into the teacher context yields only marginal student improvements. 我们在Figure 1 中展示, 若只是把原始失败结果或未结构化的环境报错直接喂给teacher, 上述做法对student 的提升非常有限。[注]6

[重点] Motivated by this observation, we propose Reflection-Enhanced Self-Distillation (RESL), a framework that enriches the teacher context with actively interpreted feedback. 基于这一观察, 我们提出Reflection-Enhanced Self-Distillation (RESL), 通过主动解释反馈来增强teacher 的上下文。For each failed rollout, RESL first generates a retrospective reflection that identifies the likely cause of failure and the correction that would have avoided it. 对于每一次失败rollout, RESL 会先生成一段回溯式反思, 判断失败的可能原因, 以及原本该如何修正才能避免这次失败。It then organizes recurring lessons into a persistent playbook, allowing future teacher prompts to reuse feedback-derived knowledge across training steps. 随后, 它会把反复出现的经验整理进一个持久化的playbook, 让后续的teacher prompt 能够跨训练步复用这些由反馈提炼出的知识。The resulting enriched teacher context transforms raw failure feedback from a passive diagnostic signal into an actionable and reusable source of corrective supervision, enabling self-distillation to improve even when successful demonstrations are unavailable. 由此得到的增强上下文, 把原本被动的失败反馈从诊断信号转化为可执行、可复用的纠错监督来源, 使自蒸馏即便在没有成功示范时也能继续提升。[注]7

Empirically, we evaluate RESL on tasks designed to reflect continuous learning beyond common post-training distributions. 在实验上, 我们把RESL 放到一组旨在体现持续学习、且超出常见后训练分布的任务上进行评估。These tasks are novel to the model, often begin in rare-success regimes, and provide rich execution feedback despite receiving sparse binary rewards. 这些任务对模型来说都是新任务, 往往从成功极少的状态起步, 同时虽然奖励只是稀疏的二元信号, 却会提供丰富的执行反馈。Across these settings, RESL substantially improves over standard self-distillation baselines and achieves faster early-stage improvement than GRPO while using only a single rollout per prompt. 在这些设定下, RESL 显著优于标准自蒸馏基线, 并且在每个prompt 只用一次rollout 的情况下, 早期提升速度也快于GRPO。These results suggest that structured feedback enables sample-efficient bootstrapping from failures, while reward-based optimization remains a complementary tool once sufficient successful rollouts are available. 这些结果说明, 结构化反馈能够让模型以更高样本效率从失败中启动学习; 而一旦成功rollout 逐渐充足, 基于奖励的优化仍然是互补且有价值的工具。[注]8

Our contributions are as follows: 本文的贡献如下:

- We identify *feedback formulation* as a critical design axis for on-policy self-distillation especially in sparse-reward regimes. 我们指出, 反馈如何被表述与组织 是on-policy 自蒸馏中的一个关键设计维度, 尤其是在稀疏奖励场景下。
- We propose *Reflection-Enhanced Self-Distillation* (RESL), which enriches the teacher context with retrospective reflections and a persistent playbook. 我们提出*Reflection-Enhanced Self-Distillation* (RESL), 通过回溯式反思与持久化playbook 来增强teacher 上下文。By interpreting failures and preserving recurring lessons across training steps, RESL turns raw feedback from a passive diagnostic signal into reusable corrective supervision. 通过解释失败并把反复出现的经验跨训练步保留下来, RESL 把原始反馈从被动诊断信号转化成可复用的纠错监督。Empirically, we characterize the sample-efficiency of feedback-enhanced self-distillation under an online streaming protocol. 在实验上, 我们在在线流式训练协议下刻画了这种反馈增强型自蒸馏的样本效率。
- More broadly, by revealing that self-improving models are highly sensitive to how feedback is structured, we position our reflection and playbook mechanisms as a plug-and-play module that can readily enhance other self-distillation objectives. 更广义地说, 我们表明自我改进型模型对反馈结构高度敏感, 因此可以把本文的reflection 与playbook 机制看成一个可插拔模块, 用来增强其他自蒸馏目标。

⁶ 【空缺|保留】
评: 这里真正把文章的切入口从“有没有反馈”推进到“反馈怎样表示”, 这是引言里最关键的一步。

⁷ 【方法|保留】
评: 方法段没有只报一个名字, 而是把reflection 和playbook 这两个动作都说清楚了, 并明确它们如何改变teacher 的上下文。

⁸ 【结果|保留】
评: 引言结尾没有把方法绝对化, 而是把structured feedback 和reward-based optimization 的分工关系交代了出来, 判断比较稳。

引言总评

这一节推进得比较完整。作者先从RL 在后训练中的通用地位起步，再指出稀疏奖励导致的启动困难，随后把OPD 与self-distillation 的已有路线快速梳理出来，最后把本文真正的新问题收敛到“反馈该如何表示与组织”。最有效的转折是从“teacher 有反馈”推进到“teacher 是否真正理解反馈”，这一步把论文的新意从常规算法替换中抽离出来。当前最主要的问题是引言里有几处句子略偏大而全，例如对structured feedback 与reward optimization 的关系判断，略早带有总结口吻。若只改一两处，优先可以压紧Figure 1 前后的论证，让“原始失败反馈为何不够用”的证据和解释衔接得更硬一些。

2 Preliminaries

预备知识

Recent advancements such as Self-Distillation Policy Optimization (SDPO) [6] and On-Policy Self-Distillation (OPSD) [30] introduce a “self-teacher” paradigm that replaces sparse outcomes in Reinforcement Learning with dense, token-level supervisions without the need for a massive separate teacher model. 近期的Self-Distillation Policy Optimization (SDPO) [6] 和On-Policy Self-Distillation (OPSD) [30] 等工作提出了“self-teacher”范式：它在不额外依赖一个庞大独立teacher 模型的前提下，把强化学习中的稀疏结果替换成稠密的token 级监督。In these frameworks, a “self-teacher” policy is typically instantiated as the model’s current or moving-average weights θ_{old} , operating in a privileged context. 在这些框架里，“self-teacher” 通常由模型当前参数或其移动平均参数 θ_{old} 来实例化，并在具备特权信息的上下文中工作。The teacher is conditioned on both the intermediate state $s_t = (x_t, y_{<t})$ and the retrospective feedback $c(x, y)$, producing a corrected, feedback-informed distribution $\pi_{\theta_{\text{old}}}(\cdot|x, y_{<t}, c)$. teacher 同时以中间状态 $s_t = (x_t, y_{<t})$ 和回溯反馈 $c(x, y)$ 为条件，生成一个带有纠错信息的分布 $\pi_{\theta_{\text{old}}}(\cdot|x, y_{<t}, c)$ 。The base policy π_{θ} is then optimized to match this privileged distribution at every decoding step. 随后，基础策略 π_{θ} 会在每个解码步上被优化去拟合这一特权分布。^{[注]9}

We formulate this matching objective universally using *f-divergences*. 我们使用*f-divergence* 来统一表述这一拟合目标。For a given time step t , let the likelihood ratio for any token $v \in \mathcal{V}$ be defined as $\tau_v^t = \frac{\pi_{\theta_{\text{old}}}(v|x, y_{<t}, c)}{\pi_{\theta}(v|x, y_{<t})}$. 对于任意时间步 t ，设任意token $v \in \mathcal{V}$ 的似然比为 $\tau_v^t = \frac{\pi_{\theta_{\text{old}}}(v|x, y_{<t}, c)}{\pi_{\theta}(v|x, y_{<t})}$ 。To prevent extreme ratios from destabilizing training, we clip the likelihood ratio to a bounded range: 为了防止极端比值破坏训练稳定性，作者把该似然比裁剪到一个有界区间内：

$$\tilde{\tau}_v^t = \text{clip}(\tau_v^t, \epsilon_{\min}, \epsilon_{\max})$$

The unified self-distillation objective then minimizes the expected f-divergence across all tokens $t \in [1, T]$: 统一后的自蒸馏目标，就是最小化所有token $t \in [1, T]$ 上的期望f-divergence:

$$\mathcal{L}_{\text{SD}}(\theta) = \mathbb{E}_{x, y \sim \pi_{\theta}, f} \left[\sum_{t=1}^T \sum_{v \in \mathcal{V}} \pi_{\theta}(v|x, y_{<t}) \cdot f(\tilde{\tau}_v^t) \right] \quad (1)$$

where $f(\tau)$ is a convex function. 其中， $f(\tau)$ 是一个凸函数。By selecting different f , we can recover various divergence measures, such as forward KL, reverse KL and Jensen-Shannon Divergence (JSD), each offering unique optimization properties. 通过选取不同的 f ，就能恢复出前向KL、反向KL 和Jensen-Shannon Divergence (JSD) 等不同散度，它们各自对应不同的优化性质。We provide a detailed discussion on the choice of f and its implications in Appendix A. 关于 f 的选择及其影响，作者在Appendix A 中给出了更详细讨论。^{[注]10}

Notice that, since we sample trajectories from the base policy π_{θ} , the self-distillation objective is inherently on-policy. 注意，由于轨迹是从基础策略 π_{θ} 中采样出来的，这个自蒸馏目标天然就是on-policy 的。Such a formulation allows the model to recover forgotten behaviors after midtraining [17, 3, 18, 10] and prevent from exposure bias [12, 2]. 这种表述方式既有助于模型在midtraining 之后恢复遗忘行为 [17, 3, 18, 10]，也能缓解exposure bias [12, 2]。

⁹ 【文献定位|保留】

评：这一段把self-teacher 框架的输入、输出和训练目标都快速交代出来，足够支撑后面的统一目标定义。

¹⁰ 【方法|保留】

评：预备知识部分把核心目标写得比较规整，也把可替换的散度族提前统一到同一个符号框架里。

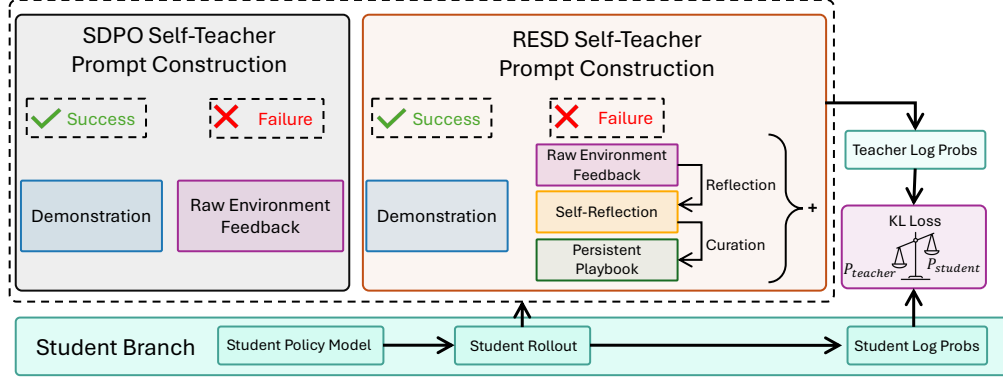


Figure 2: Overview of the RESD framework. The student generates a rollout and receives environment feedback. On failure, a local self-reflection diagnoses the error and a global playbook curation step distills reusable lessons into a persistent playbook. The enriched context (reflection, curated playbook, and any cached solutions) is fed to the teacher prompt, whose output distribution provides token-level supervision to the student via a KL loss.

预备知识总评

这一节的任务很明确，就是把后文方法所依赖的SDPO / OPSD 训练框架和统一目标函数先搭起来。作者没有在这里引入额外新概念，而是先固定self-teacher 的条件分布、似然比和clipping，再把f-divergence 的统一目标压成一个公式，组织上是清楚的。最有效的地方在于它提前说明了为什么这是on-policy 的，这能为后面讨论rare-success、failed rollout 和teacher context 的演化埋下基础。主要问题是末尾关于“恢复遗忘行为”和“防exposure bias”的表述稍显顺手一提，和本篇主线关联没有完全展开。若只改一处，优先可以把最后一句再压紧一些，让它更像是公式性质的补充，而不是额外开一条新宣传线。

3 Method

方法

Our work builds on the finding that existing self-distillation methods rely heavily on successful demonstrations, and their supervision degrades when successful rollouts are scarce. 本文建立在这样一个观察之上：现有自蒸馏方法高度依赖成功示范，而当成功rollout 稀少时，它们给出的监督质量会明显下降。We propose Reflection-Enhanced Self-Distillation (RESD), which enriches the teacher’s context with retrospective feedback from failures and a persistent memory of past experiences, enabling corrective supervision even without successful demonstrations. 我们提出Reflection-Enhanced Self-Distillation (RESD)，通过把失败回合中的回溯反馈和过往经验的持久记忆一起纳入teacher 上下文，使模型即便没有成功示范，也能获得纠错监督。We first motivate the problem by analyzing SDPO’s failure mode under sparse rewards (§3.1), then detail the retrospective reflection and playbook curation mechanism (§3.2), and describe how these components are integrated into a memory-augmented self-distillation framework (§3.3). 本节会先分析SDPO 在稀疏奖励下的失败模式 (§3.1)，再介绍回溯式反思与playbook 整理机制 (§3.2)，最后说明它们如何被整合进一个带记忆的自蒸馏框架中 (§3.3)。

3.1 Self-Distillation Relies on Successful Peer Demonstrations

自蒸馏依赖成功的同伴示范

The privileged context of the SDPO self-teacher comprises two complementary signals: environment feedback on the current trajectory and successful peer demonstrations sampled within the same rollout batch [6]. SDPO 中self-teacher 的特权上下文包含两类互补信号：当前轨迹的环境反馈，以及同一rollout batch 内采样到的成功同伴示范 [6]。To disentangle their contributions, we conduct an ablation that varies the rollout batch size N while keeping all other hyperparameters fixed. 为了拆开这两类信号各自的贡献，作者做了一个消融实验：在保持其他超参数不变的前提下，只改变rollout batch size N 。When $N=1$, each prompt group samples a single rollout, eliminating the

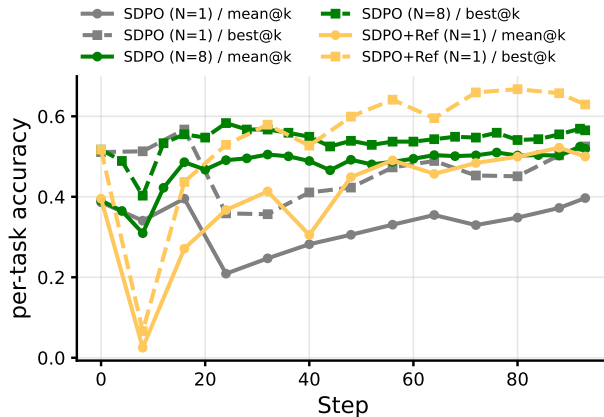


Figure 3: Per-task accuracy on FINER under varying rollout batch sizes. SDPO ($N=1$) degrades without peer demonstrations, while adding reflection and playbook curation (SDPO+Ref, $N=1$) recovers and surpasses the $N=8$ baseline.

possibility of peer demonstrations entirely and leaving environment feedback as the sole supervisory signal. 当 $N=1$ 时, 每个prompt组只采样一次rollout, 这会彻底消除同伴示范的可能性, 环境反馈也就成了唯一的监督来源。As shown in Figure 3, SDPO's performance degrades substantially in this regime, indicating that the teacher struggles to translate failure feedback alone into effective corrective distributions. Figure 3 显示, 在这一设定下SDPO 的性能会明显下降, 这说明teacher 很难只靠失败反馈就构造出有效的纠正分布。This failure mode becomes particularly severe in rare-success regime. 而在rare-success 场景中, 这种失败模式会更加严重。If the per-rollout success rate is small, most rollout groups will contain only failures and provide no positive counterfactual demonstration, forcing the teacher to construct token-level supervision from raw failure feedback alone which will result in less performance improvements through training. 如果单次rollout 的成功率很低, 那么大多数rollout group 都只会包含失败样本, 无法提供正向的反事实示范, teacher 只能从原始失败反馈中硬构造token 级监督, 于是训练期间的性能提升就会大打折扣。[注]11

3.2 From Passive Feedback Exposure to Active Feedback Understanding

从被动暴露反馈到主动理解反馈

The degradation observed in the $N=1$ setting suggests that environment feedback alone is not sufficient to support effective self-distillation. $N=1$ 设定下的退化说明, 单靠环境反馈本身并不足以支撑有效的自蒸馏。This is somewhat surprising, as the teacher is still given privileged information about the student's failed trajectory, and in principle should be able to use this information to produce a more corrective distribution. 这其实有些反直觉, 因为teacher 依然拿到了student 失败轨迹的特权信息, 按理说本该能够利用这些信息生成更具纠错性的分布。[重点] *We argue that the issue lies not in the absence of feedback but in how feedback is represented and used.* 作者认为, 问题并不在于有没有反馈, 而在于反馈是如何被表示和使用的。Existing self-distillation methods often expose the teacher to environment feedback as additional context, but leave the interpretation of that feedback entirely implicit. 现有自蒸馏方法通常只是把环境反馈作为额外上下文暴露给teacher, 却把“如何理解这些反馈”完全留给模型自己隐式处理。In rare-success regimes, this passive use of feedback becomes a bottleneck, raw failure signals are often diagnostic rather than instructional, indicating that a trajectory failed without explaining which decision caused the failure. 在rare-success 场景里, 这种被动使用反馈的方式会变成瓶颈, 因为原始失败信号往往更像诊断信息而不是指导信息, 它只说明轨迹失败了, 却不解释究竟是哪一步决策导致了失败。[注]12

This motivates a shift from passive feedback exposure to active feedback understanding. 因此, 作者提出要把范式从“被动暴露反馈”转向“主动理解反馈”。For feedback to become useful supervision, the teacher must perform two additional operations. 要让反馈真正变成有用的监督, teacher 至少还

¹¹ 【问题定义保留】

评: 这一小节先用一个很直接的ablation 把“缺成功示范时SDPO 会失灵”这件事钉住, 为后面的方法动机提供了实证支点。

¹² 【空缺保留】

评: 这里把问题进一步从“缺成功样本”推进到“失败反馈没有被解释”, 动机层面的抽象提升做得比较到位。



Figure 4: Per-prompt mean accuracy distribution on FINER over training steps. All-wrong cases’ proportion decreases better for SDPO+Ref.

需要做两件额外的事。(a) It must retrospectively interpret the failed trajectory, connecting the final feedback signal to the intermediate reasoning that caused the failure. (a) 它需要回溯式地解释失败轨迹，把最终的反馈信号和中间导致失败的推理过程连起来。(b) It must preserve useful lessons across trajectories, since many failures are repeated manifestations of the same hidden rule rather than independent mistakes. (b) 它需要把有用经验跨轨迹保留下来，因为许多失败其实是同一条隐藏规则在不同样本上的重复表现，而不是彼此独立的错误。In this view, raw feedback is only a starting point, and it must be transformed into causal explanations and persistent knowledge before it can reliably guide token-level distillation. 在这一视角下，原始反馈只是一种起点；只有当它被转化成因果解释和可持续复用的知识之后，才可能稳定地指导token级蒸馏。As shown by the yellow curve in Figure 3, equipping the teacher with only these two failure-derived components (without access to any successful demonstrations yet) already recovers much of the performance lost in the $N=1$ regime. Figure 3 中的黄色曲线表明，只要给teacher加上这两类由失败中提炼出的组件，即便仍然拿不到任何成功示范，也已经能够恢复 $N=1$ 场景中损失掉的大部分性能。Further analysis on the distribution of per-prompt accuracy (i.e. we sample 4 times for each prompt and measure the proportion of correct trials) in Figure 4 shows that the performance improvement of SDPO ($N=8$) mostly comes from increasing the number of completely correct prompts, while SDPO+Ref ($N=1$) significantly reduces the number of completely incorrect prompts, indicating that reflections help the model learn from failures. 进一步地，Figure 4 对per-prompt accuracy分布的分析（即每个prompt采样4次，并统计正确试验比例）显示，SDPO ($N=8$)的提升主要来自完全正确prompt数量的增加，而SDPO+Ref ($N=1$)则显著减少了完全错误prompt的数量，这说明reflection确实在帮助模型从失败中学习。^{[注]13}

3.3 Reflection-Enhanced Self-Distillation (RES D)

Reflection-Enhanced Self-Distillation (RES D)

[重点] Building on the above observation, we introduce Reflection-Enhanced Self-Distillation (RES D), a framework that operationalizes active feedback understanding within the self-distillation loop. 基于上述观察，作者提出Reflection-Enhanced Self-Distillation (RES D)，把“主动理解反馈”真正做进自蒸馏训练环里。RES D maintains two forms of persistent context: a playbook $\mathcal{P}_t$, following the broader idea from [29], that stores reusable lessons distilled from previous failures, and an optional solution buffer $\mathcal{B}$ that caches successful trajectories when they are available. RES D 维护两类持久上下文：其一是playbook $\mathcal{P}_t$ ，借鉴[29]的思路，用来存储从既往失败中提炼出的可复用经验；其二是可选的solution buffer $\mathcal{B}$ ，在有成功轨迹时把它们缓存下来。At each training step, RES D first updates this context using the outcome of the current rollout, and then conditions the teacher on the enriched context to produce token-level supervision. 在每个训练步中，RES D 会先根据当前rollout的结果更新这份上下文，再让teacher以增强后的上下文为条件生成token级监督。This design allows failed trajectories to improve future teacher prompts, rather than being used only as one-shot raw feedback. 这种设计使失败轨迹不再只是一次性的原始反馈，而能真正改进未来的teacher prompt。

Retrospective Reflection on Failures. For a failed trajectory (x, y) with environment feedback $c(x, y)$, RES D invokes the student model in a diagnostic mode to generate a retrospective reflection. 对于带有环境反馈 $c(x, y)$ 的失败轨迹 (x, y) ，RES D 会把student模型切换到诊断模式，生成一段回溯式反思。Formally, given the current playbook $\mathcal{P}_t$, the reflector outputs 形式化地说，在给定当前playbook $\mathcal{P}_t$ 时，reflector会输出

$$r, \{\ell_{t,j}\}_{j=1}^{|\mathcal{P}_t|} = \text{REFLECT}(x, y, c, \mathcal{P}_t; \theta),$$

¹³ [方法保留]

评：这一段既给出概念解释，也回扣了图上的实证现象，说明本文的方法不是纯直觉堆模块。

where r is a natural language explanation of the failure and $\ell_{t,j} \in \{\text{helpful}, \text{harmful}, \text{neutral}\}$ indicates whether the existing playbook entry $p_j \in \mathcal{P}_t$ helped or misled the current attempt. 其中, r 是对失败的自然语言解释, 而 $\ell_{t,j} \in \{\text{helpful}, \text{harmful}, \text{neutral}\}$ 表示现有playbook 条目 $p_j \in \mathcal{P}_t$ 对本次尝试究竟是有帮助、起误导作用, 还是中性。The reflection is designed to make the raw feedback more actionable by connecting it to specific reasoning steps. 这一反思模块的目的, 是把原始反馈和具体推理步骤对应起来, 从而让反馈更具可执行性。The entry-level tags provide a lightweight usage signal for maintaining the playbook over time. 而条目级标签则提供了一种轻量的使用信号, 用来长期维护playbook。[注]¹⁴

Playbook Curation from Past Experiences. The playbook $\mathcal{P}_t = \{p_1, \dots, p_M\}$ is a persistent set of natural-language entries that encode reusable lessons. playbook $\mathcal{P}_t = \{p_1, \dots, p_M\}$ 是一组持久存在的自然语言条目, 用来编码可复用的经验。Given a reflection r , the curator generates new entries to add: 给定一段reflection r 之后, curator 会生成新的条目加入其中:

$$\mathcal{P}_{t+1} = \mathcal{P}_t \cup \text{CURATE}(\mathcal{P}_t, r; \theta)$$

where π_θ is prompted with the current playbook and the reflection, and produces a set of non-redundant entries that capture lessons not already present in $\mathcal{P}_t$. 其中, π_θ 会以当前playbook 和reflection 为条件, 被提示去生成一组不冗余的新条目, 用来记录 $\mathcal{P}_t$ 中尚未包含的经验。Each entry p_j maintains accumulated counts (h_j, d_j) of helpful and harmful tags received from the reflector across steps. 每个条目 p_j 都会维护一组累计计数 (h_j, d_j) , 表示它在不同训练步里分别被reflector 标成helpful 和harmful 的次数。Before each context update, a CONCISE operation prunes the playbook by removing entries where $d_j \geq h_j$ (net harmful) and, if $|\mathcal{P}_t|$ exceeds a budget $M_{\max}$, evicting the least recently tagged entries. 在每次上下文更新前, 系统会先执行一次CONCISE 操作: 删除满足 $d_j \geq h_j$ 的条目 (即净有害条目); 如果 $|\mathcal{P}_t|$ 超过预算 $M_{\max}$, 还会继续驱逐最近最少被标记的条目。This lifecycle ensures the playbook remains compact and converges toward entries that are repeatedly useful across trajectories. 这套生命周期设计保证playbook 保持紧凑, 并逐渐收敛到那些在不同轨迹里反复被证明有用的经验条目上。We detail the full CONCISE strategy, including trigger conditions and two removal variants, in Appendix D. 关于完整的CONCISE 策略, 包括触发条件和两种删除变体, 作者在Appendix D 中给出了详细说明。[注]¹⁵

Interleaving Context Update with Policy Optimization. Algorithm 1 summarizes the full RESD training loop. Algorithm 1 总结了RESD 的完整训练循环。At each step t , the student π_θ generates a rollout $y \sim \pi_\theta(\cdot | x)$ and receives environment feedback $c(x, y)$ and reward $R(x, y)$ to judge the correctness. 在每个训练步 t , student π_θ 会先生成一个rollout $y \sim \pi_\theta(\cdot | x)$, 并接收环境反馈 $c(x, y)$ 和奖励 $R(x, y)$ 来判断其正确性。Before the policy update, a context update phase modifies the persistent state: CONCISE first prunes $\mathcal{P}_t$, then, if the rollout failed, REFLECT and CURATE extend the playbook to $\mathcal{P}_{t+1}$; if it succeeded, the solution buffer is updated $\mathcal{B}(x) \leftarrow y$ and sets $r = \emptyset$. 在策略更新之前, 会先进入上下文更新阶段: CONCISE 先裁剪 $\mathcal{P}_t$; 如果rollout 失败, 就调用REFLECT 和CURATE 把playbook 扩展为 $\mathcal{P}_{t+1}$; 如果rollout 成功, 则更新solution buffer $\mathcal{B}(x) \leftarrow y$, 并令 $r = \emptyset$ 。The teacher model is then synced with student model with an EMA update [6], and conditioned on the enriched context to produce token-level supervision: 随后, teacher 模型通过一次EMA 更新与student 同步 [6], 并以增强后的上下文为条件生成token 级监督:

$$\pi_{\theta_{\text{old}}}(\cdot | x, y_{<t}, c, r, \mathcal{P}_{t+1}, \mathcal{B}(x))$$

and the student minimizes the self-distillation objective $\mathcal{L}_{\text{SD}}(\theta)$ (Eq. 1). 而student 则最小化自蒸馏目标 $\mathcal{L}_{\text{SD}}(\theta)$ (Eq. 1)。We additionally apply a per-sample weight to enhance the gradient contribution of successful samples, computed as a function of the batch success rate (details in Appendix B). 作者还额外引入了逐样本权重, 用来增强成功样本的梯度贡献; 该权重由batch success rate 决定 (细节见Appendix B)。Because $\mathcal{P}$ and $\mathcal{B}$ persist across steps, the supervision available to the teacher can improve over time even when the current batch contains no successful demonstration. 由于 $\mathcal{P}$ 和 $\mathcal{B}$ 都会跨训练步持续存在, 即便当前batch 里没有成功示范, teacher 能用到的监督信息也仍然可能随时间不断改善。[注]¹⁶

¹⁴ 【方法保留】

评: 反思模块这里不是只说“生成一段reflection”, 而是把输出结构细化成解释文本和条目标签, 设计意图比较清楚。

¹⁵ 【方法保留】

评: playbook 这一段最有效的地方是没有把memory 写成黑箱, 而是明确给出了新增、计数、裁剪和预算控制四个环节。

¹⁶ 【方法保留】

评: 最后这段把context update 和policy optimization 如何交错运行讲清楚了, 也解释了为什么失败经验能够跨步积累成真正的监督增益。

Algorithm 1 Reflection-Enhanced Self-Distillation (RESL)

Require: Student policy π_θ , teacher weights θ_{old} , playbook $\mathcal{P} \leftarrow \emptyset$, solution buffer $\mathcal{B} \leftarrow \emptyset$

```
1: for each training step do
2:   // Rollout & Reward
3:   Sample rollout  $y \sim \pi_\theta(\cdot | x)$ 
4:   Obtain environment feedback  $c(x, y)$  and reward  $R(x, y)$ 
5:   // Context Update
6:    $\mathcal{P} \leftarrow \text{CONCISE}(\mathcal{P})$  ▷ prune stale/harmful entries
7:   if  $R(x, y) < \text{threshold}$  then
8:      $r \leftarrow \text{REFLECT}(x, y, c, \mathcal{P}; \theta)$  ▷ generate reflection and tag playbook entries
9:      $\mathcal{P} \leftarrow \text{CURATE}(\mathcal{P}, r; \theta)$  ▷ generate new playbook entries from reflection
10:  else
11:     $\mathcal{B}(x) \leftarrow y$  ▷ cache successful solution for future replay
12:  end if
13:  // Enriched Teacher Prompt Construction
14:  Retrieve buffered solution  $\mathcal{B}(x)$  (if available)
15:  Construct enriched context: playbook  $\mathcal{P}$ , reflection  $r$ , previous trial  $y$ , feedback  $c$ , solution  $\mathcal{B}(x)$ 
16:  // Policy Update
17:  Compute teacher distribution  $\pi_{\theta_{\text{old}}}(\cdot | x, y_{<t}, c, r, \mathcal{P}, \mathcal{B}(x))$ 
18:  Update  $\theta$  by minimizing  $\mathcal{L}_{\text{SD}}(\theta)$ 
19:   $\theta_{\text{old}} \leftarrow \theta$  ▷ sync teacher weights
20: end for
```

方法总评

方法部分的组织是清楚的：先用一个ablation说明SDPO在缺少成功示范时为什么会失效，再把失败反馈需要被“解释”和“积累”这两个核心动作抽象出来，最后再落到具体的reflection、playbook和训练循环。最有效的地方是它没有把memory写成泛泛的经验池，而是把条目标签、累计计数、裁剪预算和context update次序这些机制都落实到了可执行层。当前最主要的问题是第二小节到第三小节之间有一点概念密度偏高，尤其在第一次引入playbook生命周期时，读者需要同时跟踪多个符号和模块职责。若只改一两处，优先可以把REFLECT与CURATE的接口差异再显式说一句，让“解释失败”和“提炼长期规则”的边界更一眼可见。

4 Experiments

实验

Our experiments are designed to address the following questions: 本文的实验主要围绕以下几个问题展开：

- Does RESL improve over self-distillation baselines especially in rare-success regimes? (§4.2, §4.3) RESL 是否能够在rare-success场景下显著优于自蒸馏基线？ (§4.2, §4.3)
- How does RESL compare to reward-based RL (GRPO) in terms of interaction efficiency? (§4.4) 在交互效率上，RESL与基于奖励的RL (GRPO) 相比表现如何？ (§4.4)
- How does each component of RESL affect the performance? (§4.5) RESL的各个组件分别如何影响性能？ (§4.5)
- How does RESL interpret and organize failure feedback at the instance level? (§4.6) 在单个实例层面上，RESL是怎样解释并组织失败反馈的？ (§4.6)

We additionally evaluate instruction-following preservation and training latency in Appendices E and F. 此外，作者还在Appendices E和F中补充评估了instruction-following保持情况与训练时延。

4.1 Experimental Setup

实验设置

Datasets. We evaluate on four tasks spanning program synthesis and physical reasoning. 作者在四个横跨程序合成和物理推理的任务上进行评估。MANUFACTORIA-HAS and BOUNCINGSIM-EASY/MEDIUM are drawn from the RL-Grok benchmark [20], while FINER is from [29].

Table 1: Overview of evaluation tasks. “Train” and “Test” denote the number of unique problems in each split.

Task	Train	Test	Description
MANUFACTORIA-HAS [20]	742	132	Write DSL programs that check whether an input tape contains a target pattern
BOUNCINGSIM-EASY [20]	640	100	Write Python code to simulate 2-D multi-object bouncing dynamics (easy)
BOUNCINGSIM-MEDIUM [20]	320	100	Write Python code to simulate 2-D multi-object bouncing dynamics (medium)
FINER [29]	1000	500	Tag financial named entities in SEC filings with XBRL labels

MANUFACTORIA-HAS 与BOUNCINGSIM-EASY/MEDIUM 取自RL-Grok benchmark [20], 而FINER 则来自 [29]。The RL-Grok tasks are characterized by near-zero initial success rates, making them a natural testbed for learning from failure feedback. RL-Grok 系列任务的特点是初始成功率几乎为零, 因此非常适合用来检验“能否从失败反馈中学习”。FINER, by contrast, exhibits a higher initial success rate, allowing us to assess whether RESD also benefits regimes where successful demonstrations are more accessible. 相比之下, FINER 的初始成功率更高, 因此可以用来检验: 当成功示范更容易得到时, RESD 是否依然有帮助。Table 1 summarizes the datasets. Table 1 对这些数据集做了概览总结。We train Qwen3-4B-Thinking-2507 for MANUFACTORIA-HAS, BOUNCINGSIM-EASY, and FINER, and Qwen3-30B-A3B-Thinking-2507 for the harder BOUNCINGSIM-MEDIUM. 在模型选择上, 作者对MANUFACTORIA-HAS、BOUNCINGSIM-EASY 和FINER 使用Qwen3-4B-Thinking-2507, 对更难的BOUNCINGSIM-MEDIUM 使用Qwen3-30B-A3B-Thinking-2507。

Settings. We adopt an *online streaming* training protocol in which the model makes a single pass over the training data and each training example is seen at most once. 作者采用一种*online streaming* 训练协议: 模型只对训练数据做单次遍历, 每个训练样本至多被看到一次。For every incoming batch, the trainer executes an inner loop of up to $K = 4$ update iterations on the same set of prompts. 对于每个新到来的batch, trainer 会在同一组prompts 上执行最多 $K = 4$ 次内循环更新。This protocol is designed to mimic practical self-improvement scenarios, where deployed models continuously encounter new tasks, observe task outcomes, and must update under a limited interaction budget ($N=1$). 这一协议旨在模拟真实的自我改进场景: 部署中的模型会持续遇到新任务、观察任务结果, 并且只能在有限的交互预算下更新自己 (这里是 $N=1$)。Unless otherwise specified, we adopt EMA update for teacher model weights with an update rate of 0.0001. 除非另有说明, teacher 模型参数都通过EMA 更新, 更新率设为0.0001。We optimize reverse-KL divergence as the self-distillation loss l with a lower clip $\epsilon_{\min} = 0.2$ and no upper clip, under train data batch size of 32. 自蒸馏损失使用reverse-KL (见 1), 下界裁剪设为 $\epsilon_{\min} = 0.2$, 上界不裁剪, 训练batch size 为32。We provide further implementation details in Appendix G. 更多实现细节见Appendix G。

Metrics and Baselines. We report both *mean@4* ($m@4$), the average performance over four independently sampled responses per test problem, and *best@4* ($b@4$), the best performance among the four samples. 作者同时报告*mean@4* ($m@4$) 和*best@4* ($b@4$): 前者是每个测试样本独立采样四次后的平均表现, 后者是四次采样中的最佳表现。We use two evaluation metrics: *Per-Task Accuracy*, which measures the fraction of problems whose response passes all test cases (equivalent to $\text{pass}@1$ for a single sample, also used as reward function), and *Per-Test-Case Accuracy*, which measures the average fraction of test cases passed across problems. 评测指标有两个: *Per-Task Accuracy* 衡量回答完全通过全部测试用例的问题占比 (对单次采样来说等价于 $\text{pass}@1$, 也被用作奖励函数); *Per-Test-Case Accuracy* 衡量跨问题平均通过的测试用例比例。Since different validation metrics may peak at different training steps, we select a single checkpoint per run using a rank-based rule: for each logged step, we rank its four validation metrics, sum the ranks, and report the checkpoint with the highest total rank. 由于不同验证指标的峰值可能出现在不同训练步上, 作者采用一种基于排名的规则来给每次运行选择唯一checkpoint: 对每个记录步的四个验证指标分别排序, 再把名次求和, 最终选总名次最高的checkpoint。The same selection rule is applied uniformly to all methods. 这一checkpoint 选择规则对所有方法都统一适用。We additionally include **SDPO+ss**, a variant that also distills on the self-success instead of skipping them (disabled as a default behavior in SDPO), providing denser teacher supervision in higher-success regimes. 作者还额外加入了**SDPO+ss** 作为对比, 它不会像标准SDPO 那样跳过self-success, 而是也在这些样本上蒸馏, 从而在较高成功率场景下提供更稠密的teacher 监督。^{[注]17}

4.2 Main Results

主要结果

¹⁷ 【方法保留】

评: 实验设置写得比较实, 不只是报超参数, 还解释了streaming protocol 为什么能对应真实自我改进场景。

Table 2: Main results on four tasks. $m@4$ and $b@4$ denote mean@4 and best@4. Per-Task Accuracy is the fraction of problems whose response passes all test cases; Per-Test-Case Accuracy is the average fraction of test cases passed.

Method	MANUFACTORIA-HAS (Qwen3-4B)		BOUNCINGSIM-EASY (Qwen3-4B)		BOUNCINGSIM-MEDIUM (Qwen3-30B-A3B)		FINER (Qwen3-4B)	
	$m@4$	$b@4$	$m@4$	$b@4$	$m@4$	$b@4$	$m@4$	$b@4$
<i>Per-Task Accuracy</i>								
Base model	0.38	1.52	0.00	0.00	1.00	3.00	39.28	51.90
SDPO [6]	0.95	3.79	1.00	3.00	2.00	6.00	40.93	50.50
SDPO+ss	0.57	2.27	1.00	4.00	2.00	6.00	50.25	59.12
RESL (ours)	35.80	65.91	4.25	8.00	7.25	10.00	53.66	61.12
<i>Per-Test-Case Accuracy</i>								
Base model	25.31	60.97	27.85	33.53	23.72	34.67	69.65	79.21
SDPO [6]	37.66	73.55	28.98	46.20	24.38	34.47	71.27	78.21
SDPO+ss	37.97	71.93	29.68	47.73	24.93	35.80	76.15	82.21
RESL (ours)	76.95	96.53	38.87	55.80	34.72	41.07	78.98	84.32

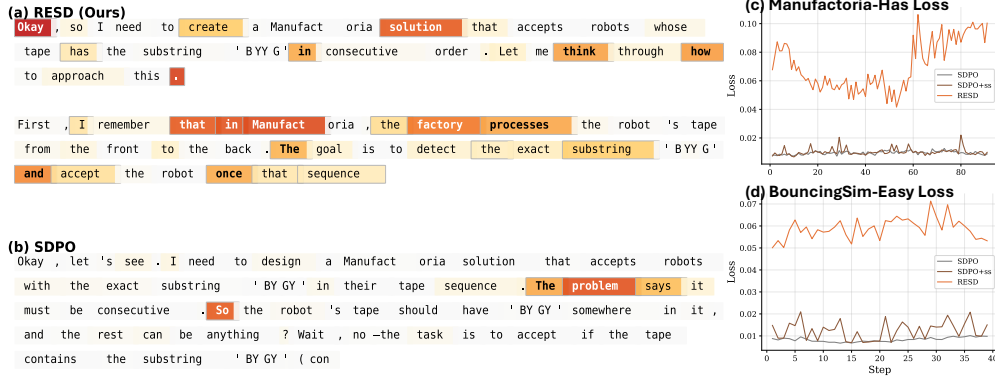


Figure 5: (a)(b) Token-level distillation loss for training step 20. Each token is shaded from white (zero loss) to dark red (high loss). (c)(d) Average per-token distillation loss across training steps for RESL and baselines.

Table 2 reports the main results across all four tasks. Table 2 汇报了四个任务上的主要实验结果。[重点]RESL achieves the best performance across all tasks and metrics. RESL 在所有任务和所有指标上都取得了最好的表现。The gains are largest in the rare-success regimes. 而且, 这些增益在rare-success 场景中最为明显。On MANUFACTORIA-HAS, RESL improves per-task accuracy from 0.57/2.27 under SDPO+ss to 35.80/65.91 in $m@4/b@4$, while also improving per-test-case accuracy from 37.97/71.93 to 76.95/96.53. 以MANUFACTORIA-HAS 为例, RESL 把per-task accuracy 从SDPO+ss 的0.57/2.27 提升到35.80/65.91 ($m@4/b@4$), 同时也把per-test-case accuracy 从37.97/71.93 提升到76.95/96.53。RESL also consistently improves over SDPO and SDPO+ss on both BOUNCINGSIM tasks, showing that failure-derived reflection and playbook memory are useful beyond the DSL setting. 在两个BOUNCINGSIM 任务上, RESL 同样稳定超过SDPO 和SDPO+ss, 说明由失败中提炼出的reflection 和playbook memory 并不只对DSL 任务有效。On FINER, where successful demonstrations are more accessible, RESL still obtains the strongest results, suggesting that structured feedback remains beneficial even outside the most sparse-success regime. 即便在成功示范更容易获得的FINER 上, RESL 依然拿到了最强结果, 这说明structured feedback 的价值并不限于最极端的sparse-success 场景。Full validation training curves for every run in Table 2 are provided in Appendix C. Table 2 中各次运行的完整验证曲线见Appendix C。[注]18

4.3 Reflection and Playbook Elicit a More Corrective Teacher

Reflection 与Playbook 让Teacher 更具纠错性

We next examine how enriching the teacher prompt changes the self-distillation signal. 接下来, 作者进一步分析增强teacher prompt 之后, 自蒸馏信号究竟发生了怎样的变化。Figure 5 plots both per-

¹⁸ [结果保留]

评: 主结果段没有只给“全面更优”的结论, 而是抓住了rare-success 和跨任务泛化这两个最重要的解释点。

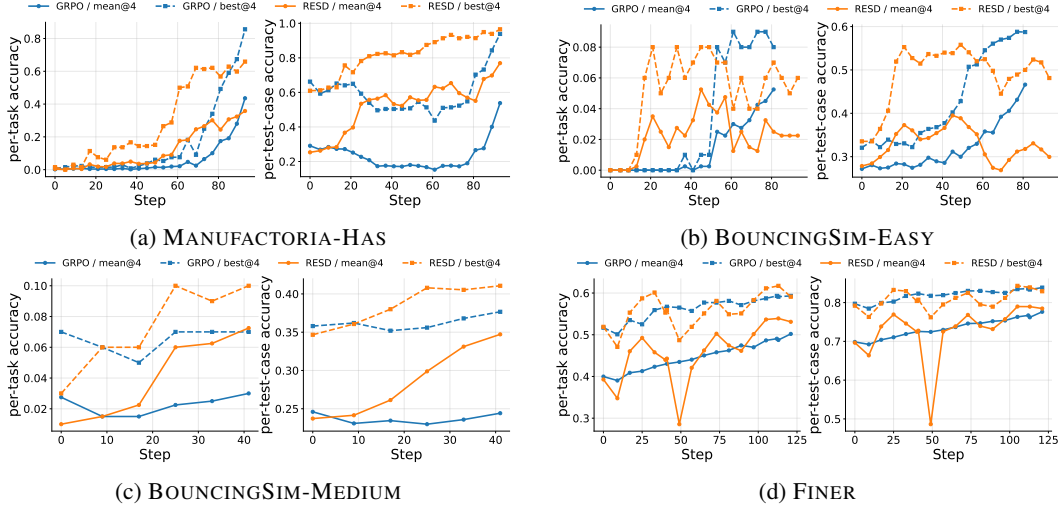


Figure 6: Validation performance across training steps for RESD and GRPO. GRPO samples 8 rollouts per prompt, while RESD uses a single rollout.

token and average per-token distillation losses over training steps. Figure 5 给出了训练过程中逐token的distillation loss 以及其平均值。(1) Across tasks, RESD often yields a higher self-distillation loss than the baselines. (1) 在不同任务上, RESD 往往会得到比基线更高的self-distillation loss。Since this loss measures the discrepancy between the student distribution and the feedback-conditioned teacher distribution, a higher value suggests that the reflection- and playbook-augmented teacher provides a distribution that deviates more from the student’s current behavior. 由于这一损失衡量的是student 分布与带反馈条件的teacher 分布之间的差异, 因此更高的数值意味着: 加入reflection和playbook 之后, teacher 给出的分布更明显地偏离了student 当前的行为。(2) RESD has more loss mass focusing on reasoning decision tokens, like “create”, “solution”, “processes” and “The goal”. (2) RESD 的loss mass 更多集中在推理决策类token 上, 例如“create”、“solution”、“processes”和“The goal”。These are points where the models choose what to do next or how to frame the problem, which demonstrate that the teacher is correcting the reasoning strategy. 这些token 恰好对应模型决定“下一步做什么”或“该如何框定问题”的位置, 这说明teacher 在纠正的不是表层字词, 而是推理策略本身。[注]19

4.4 Comparison with GRPO

与GRPO 的比较

We compare RESD with GRPO [16], a strong on-policy reinforcement learning baseline, in Figure 6. Figure 6 将RESD 与一个强势的on-policy 强化学习基线GRPO [16] 进行了比较。The key difference is the interaction pattern: GRPO uses grouped rollouts with group size 8, requiring eight sampled trajectories and reward/feedback evaluations per prompt, whereas RESD uses a single rollout per prompt and updates its teacher context from that trajectory. 两者最核心的差别在于交互模式: GRPO 使用group size为8的grouped rollouts, 因此每个prompt 都需要采样八条轨迹并做八次奖励/反馈评估; 而RESD 每个prompt 只使用一条rollout, 并直接从这条轨迹里更新teacher context。We therefore view this comparison as an interaction-efficiency comparison rather than a strict equal-rollout-budget comparison. 因此, 作者把这里的比较理解为交互效率比较, 而不是严格意义上的等rollout 预算比较。Despite using fewer rollouts per prompt, RESD improves faster than GRPO in the early stage of training across the tasks in Figure 6. 即便每个prompt 使用的rollout 更少, RESD 在Figure 6 展示的多个任务上, 训练早期仍然比GRPO 提升得更快。These results suggest that RESD is well suited for sample-efficient bootstrapping under limited interaction budgets. 这说明RESD 很适合在交互预算受限时, 以较高样本效率完成冷启动。We note that RESD does not always improve monotonically and the validation curves exhibit occasional dips. 当然, RESD 的改进并不总是单调的, 验证曲线中也会出现一些下探。On FINER, the sharp drop around step 50 is primarily caused by response truncation. 在FINER 上, step 50 左右的明显下跌主要是由response truncation 引起的。On BOUNCINGSIM-EASY, the later-stage degradation after around step 60 suggests that once the model reaches a higher-success

¹⁹ 【结果保留】

评: 这一小节有效地把方法优势从最终准确率推进到训练信号形态上, 解释了为什么structured feedback 会更有用。

Table 3: Ablation of teacher prompt components. We report per-test-case accuracy (m@4 / b@4).

Method	MANUF.-HAS		BSIM-EASY		FINER	
	m@4	b@4	m@4	b@4	m@4	b@4
RES D (full)	76.95	96.53	38.87	55.80	78.98	84.32
w/o reflection	51.41	79.26	36.85	55.33	77.78	82.26
w/o playbook	60.65	92.37	32.95	53.60	77.42	83.07
w/o solution buffer	60.24	80.10	31.97	52.40	78.36	83.42

regime, dense self-distillation from feedback-derived contexts becomes less stable than reward-based optimization. 而在BOUNCINGSIM-EASY上, step 60 之后的后期退化则提示: 一旦模型进入较高成功率阶段, 基于反馈上下文的稠密自蒸馏可能会变得不如reward-based optimization 稳定。A practical strategy is to use RES D to quickly reach a higher-success regime, then switch to stabler reward-based optimization such as GRPO once grouped rollouts provide reliable learning signals. 因此, 一个更实用的策略可能是: 先用RES D 快速把模型推到较高成功率区域, 再在grouped rollouts 能提供稳定学习信号之后, 切换到更稳的reward-based optimization, 比如GRPO。[注]20

4.5 Ablation Study on Teacher Prompt Components

Teacher Prompt 组件消融

We ablate RES D’s enriched teacher context by removing local reflection, persistent playbook, or solution buffer. 作者通过分别去掉local reflection、持久化playbook 和solution buffer, 对RES D 的增强teacher context 做了消融。Table 3 shows that full RES D achieves the strongest performance across all three tasks. Table 3 表明, 完整的RES D 在三个任务上都取得了最强表现。Removing reflection causes the largest degradation on MANUFACTORIA-HAS (76.95→51.41 m@4), while removing the playbook or solution buffer also lowers performance, suggesting that local reflection provides useful short-term corrections while the playbook preserves reusable lessons and prevents regressions. 去掉reflection 会在MANUFACTORIA-HAS 上带来最大幅度的退化 (m@4 从76.95 降到51.41); 而去掉playbook 或solution buffer 也都会造成性能下降, 这说明local reflection 提供的是短期纠错能力, 而playbook 负责保留可复用经验、避免回退。The solution buffer further helps convert partial progress into fully correct solutions. solution buffer 则进一步帮助模型把“部分做对”推进为“完整做对”。

4.6 Case Study: How RES D Interprets Failure Feedback

案例分析: RES D 如何解释失败反馈

Table 4 traces RES D on a representative MANUFACTORIA-HAS task across four consecutive inner-loop updates, illustrating how local environment feedback is progressively organized into reusable task knowledge. Table 4 跟踪了RES D 在一个代表性的MANUFACTORIA-HAS 任务上的四次连续inner-loop 更新, 展示了局部环境反馈如何被逐步组织成可复用的任务知识。This case highlights two mechanisms that are difficult to see from aggregate accuracy alone. 这个案例突出了两个仅看总体准确率很难看清的机制。[重点]First, RES D exposes learning progress before sparse success appears: the model fixes syntax, then transition handling, then a more subtle edge case involving empty-tape behavior. 第一, RES D 能在稀疏成功真正出现之前, 就显露出学习进展: 模型先修好语法, 再修正状态转移处理, 最后解决一个与空tape 行为相关的更细微边界情况。[重点]Second, the playbook accumulates situation-specific rules: different failure modes require different corrections (e.g., skipping non-pattern symbols vs. rejecting when the tape is empty), and the playbook records each rule alongside the situation it applies to. 第二, playbook 会积累情境化规则: 不同失败模式需要不同修正 (例如“跳过不相关符号”和“在tape 为空时拒绝”不是一回事), 而playbook 会把每条规则和它适用的情形一起记下来。A cached successful exemplar ultimately demonstrates how these rules coexist in a complete working program. 最终, 被缓存下来的成功示例则展示了这些规则如何在一个完整可运行程序中共同发挥作用。This supports the view that failure feedback becomes useful not merely by being appended to the prompt, but by being organized into persistent, scoped knowledge. 这进一步支持了本文的核心判断: 失败反馈之所以会变得有用, 并不是因为它被简单拼接进prompt, 而是因为它被组织成了持久且有作用域边界的知识。[注]21

²⁰ 【结果|保留】

评: 这一段没有把比较讲成“谁全面压倒谁”, 而是把interaction efficiency 和后期稳定性分开讨论, 结论更可信。

²¹ 【结果|保留】

Table 4: Case study on MANUFACTORIA-HAS: “Accept if the tape contains BRBR.” In this task, the model must write a finite-state machine program in a domain-specific language that processes an input tape—a sequence of colored symbols (e.g., GRBRBRY, BRGBY)—one symbol at a time from left to right. Each state reads the next symbol and branches to a successor state depending on the symbol; a tape is accepted by reaching a designated accept state and rejected otherwise. Here, the target program must accept any tape containing the contiguous substring BRBR (e.g., accept GRBRBRY, reject BRGBY), requiring the model to handle arbitrary surrounding symbols, restart matching after partial mismatches, and correctly reject when the input ends mid-match. “acc.” denotes per-test-case accuracy. Each row corresponds to one inner-loop update step, showing how RESD progressively diagnoses and corrects errors through reflection and playbook accumulation. Full generated programs for each step are provided in Appendix H.

Step	acc.	Failure reason	Reflection / playbook update
45	0.00	Program references a non-existent state, causing a parse error	Reflection extracts a syntax rule: all branch targets must reference states that are explicitly declared in the program.
46	0.84	Incorrectly rejects tapes whose leading symbols are not part of the pattern (e.g., rejects GBRBR because it starts with G, even though BRBR appears later)	Reflection identifies that state s0 routes a leading G to reject instead of continuing the search. Playbook records: non-pattern symbols before or between matches should be skipped, not rejected.
47	0.96	Overcorrection: the fix for skipping non-pattern symbols introduces an infinite loop between two states when the tape becomes empty at a non-terminal state	Reflection identifies that non-pattern symbols should be skipped when the tape is non-empty, but the program should reject when the tape is empty at a non-terminal state. The playbook records both situation-specific rules.
48	1.00	—	A successful solution is cached.

实验总评

实验部分的优点在于问题意识非常明确：先给出streaming 协议下的整体结果，再解释teacher 信号为何更具纠错性，再和GRPO 做交互效率层面的比较，最后用消融和case study 把组件作用拆开。最有效的是主结果和GRPO 对比都没有停留在“数值更高”，而是明确把rare-success、单次rollout 和后期稳定性这些边界一并说清楚。当前最主要的问题是实验节的信息量比较大，尤其从loss analysis 切到GRPO、再切到ablation 时，阅读节奏略显紧。若只改一两处，优先可以在各小节开头再加一句过渡，明确这一小节是在回答开头列出的哪一个研究问题。

5 Related Works

相关工作

On-policy self-distillation. Knowledge distillation is widely used as an alternative to supervised fine-tuning (SFT) when a stronger teacher model is available. 当存在一个更强的teacher 模型时，知识蒸馏常常被用作supervised fine-tuning (SFT) 的替代方案。Classical approaches train a student to match the output distribution or representations of a teacher [8, 15, 21, 9]. 经典做法会训练student 去拟合teacher 的输出分布或中间表征 [8, 15, 21, 9]。However, standard distillation relies on fixed data and suffers from student-teacher mismatch, since the student is evaluated on its own generated trajectories [1, 24, 4]. 但标准蒸馏依赖固定数据，而且会遭遇student-teacher mismatch，因为student 最终要在它自己生成的轨迹上被评估 [1, 24, 4]。To reduce this distribution shift, recent works study on-policy distillation [12, 23, 25, 5], where the student learns from teacher feedback on its own generations. 为了减轻这种分布偏移，近期工作开始研究on-policy distillation [12, 23, 25, 5]，让student 直接从teacher 针对自身生成结果给出的反馈中学习。This aligns the training distribution with the student policy and is closely related to online imitation learning. 这使训练分布与student 策略本身保持一致，也让这类方法与online imitation learning 很接近。Building on this idea, self-distillation uses

评：案例分析很好地把aggregate metrics 看不到的渐进式纠错过程具体展开了，也让playbook 的“情境化规则”作用变得可见。

a self-teacher that has access to privileged information to guide the current policy. 在这一思路上进行一步发展后，自蒸馏会使用一个可以访问特权信息的self-teacher 来指导当前策略。The teacher may condition on environment feedback or execution signals [6, 27], or leverage reference solutions and hints [30, 2]. 这个teacher 既可以以环境反馈或执行信号为条件 [6, 27]，也可以利用参考解答和提示信息 [30, 2]。Other works explicitly study privileged information distillation in this setting [14, 3]. 还有一些工作则更明确地研究了这一设定下的特权信息蒸馏 [14, 3]。While these methods are restricted to isolated instance-level corrections, our approach introduces cross-sample reflection to aggregate experience and provide a richer training signal. 但这些方法大多仍局限在单个样本内的局部纠正，而本文的方法则通过跨样本的reflection 去积累经验，从而形成更丰富的训练信号。[注]22

Continual learning via self-reflection and experience accumulation. Recent work explores training-free continual learning by extracting reusable strategies from an LLM’s own trajectories. 最近也有不少工作探索不依赖训练的持续学习，通过从LLM 自身轨迹中提炼可复用策略来实现改进。Models can utilize verbal feedback as episodic memory [19, 13], self-discover reasoning structures [32, 22], or maintain an evolving experience pool to dynamically inject refined strategies into future inference [29, 28]. 模型可以把verbal feedback 当作情节记忆 [19, 13]，也可以自行发现推理结构 [32, 22]，或者维护一个持续演化的经验池，把提炼后的策略动态注入后续推理 [29, 28]。Building on this trajectory-based memory paradigm, our method integrates experience accumulation directly into on-policy self-distillation. 本文正是在这种基于轨迹记忆的范式上，进一步把经验累积直接整合进on-policy self-distillation 训练环中。

相关工作总评

相关工作部分虽然很短，但没有失去主线。第一段围绕distillation 的数据分布与teacher 信息来源展开，第二段再把trajectory memory、self-reflection 和经验池类工作接进来，最后都指向本文的“跨样本反思+ 持久经验累积”定位。最有效的是它把本文与已有self-distillation 工作的差异放在“监督信号的组织范围”上，而不是只说多加了两个模块。主要不足是第二段略压缩，training-free continual learning 与本文这种训练内更新之间的张力还可以再解释一句。若只改一处，优先补一句说明本文并不是把这些memory 方法原样搬来，而是把它们改造成teacher context 的训练期组成部分。

6 Discussion and Conclusions

讨论与结论

In this work, we address the challenge of continuous LLM self-improvement via environmental interactions, particularly in sparse-reward regimes. 本文聚焦于一个问题：如何让LLM 通过环境交互持续自我改进，尤其是在奖励稀疏的场景中。We introduce Reflection-Enhanced Self-Distillation (RES D), a framework that utilizes retrospective reflections and a persistent playbook to successfully transform raw failure feedback into active, corrective supervision. 我们提出Reflection-Enhanced Self-Distillation (RES D)，利用回溯式反思和持久化playbook，把原始失败反馈转化成主动的纠错监督。Empirical evaluations across multiple complex reasoning tasks demonstrate that RES D significantly enhances interaction efficiency. 在多个复杂推理任务上的实验表明，RES D 能显著提升交互效率。By relying on only a single rollout per prompt, it achieves rapid early-stage convergence, substantially outperforming standard self-distillation methods and GRPO. 它只依赖每个prompt 一次rollout，就能在训练早期快速收敛，并明显超过标准自蒸馏方法和GRPO。[注]23

RES D as a feedback-enhancement module. The non-monotonic validation curves in Figure 6 highlight an important property of on-policy self-distillation: its effectiveness depends not only on the structure and reliability of the teacher context, but also on the optimization objective. Figure 6 里那些并不单调的验证曲线揭示了on-policy self-distillation 的一个重要性质：它的效果不仅取决于teacher context 的结构和可靠性，也取决于优化目标本身。Recent studies similarly observe that self-distillation can exhibit unstable dynamics [7, 11, 26]. 近期也有研究观察到，自蒸馏过程本身可能呈现出不稳定动力学 [7, 11, 26]。Our results complement this line of work from a different angle. 本文的结果从另一个角度补充了这条研究线。Rather than proposing a new self-distillation objective,

²² 【文献定位保留】

评：第一段related work 不是简单罗列，而是沿着distillation 分布失配这一主线逐步收束到本文的差异点，结构是有效的。

²³ 【结论保留】

评：结论开头较好地吧任务、方法和主要实验结论收拢在一起，没有再引入新设定。

RES D improves the information supplied to the self-teacher by converting raw failure feedback into local reflections and persistent playbook knowledge. 我们并没有重新设计一个新的自蒸馏目标，而是通过把原始失败反馈转化为局部反思和持久化playbook 知识，去改善self-teacher 实际拿到的信息。This explains why RES D is especially effective in the early rare-success regime, where reward-only methods obtain little signal and standard self-distillation lacks successful demonstrations. 这也解释了为什么RES D 在早期成功极少的阶段尤其有效：那时纯奖励方法拿不到足够信号，而标准自蒸馏又缺乏成功示范。More broadly, RES D can be plugged into stronger self-distillation algorithms: sample routing can decide when to apply distillation [11], reward-grounded objectives can determine update directions [26], and RES D can provide the structured feedback context that makes failed trajectories useful for token-level supervision. 更广义地说，RES D 可以被插入更强的自蒸馏算法中：sample routing 决定何时蒸馏 [11]，reward-grounded 目标决定往哪个方向更新 [26]，而RES D 则提供让失败轨迹也能变得对token 级监督有用的结构化反馈上下文。[注]²⁴

结论总评

结论部分完成了两件事：先把本文的任务、方法和主要实验结果收束回来，再进一步把RES D 解释成一个可插拔的feedback-enhancement module。第二步是这一节最有效的地方，因为它没有重复实验表，而是把本文放回更宽的self-distillation 设计空间里。主要问题是最后一段的外推稍强，尤其是“能接到更强算法上”这一点目前主要还是概念上的可兼容，而不是本文直接验证过的事实。若只改一处，优先可以把最后一句的口气再压稳一点，明确它是一个由当前结果支持的设计推论，而不是已经被全面验证的结论。

References

- [1] Rishabh Agarwal, Nino Vieillard, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. Gkd: Generalized knowledge distillation for auto-regressive sequence models. *arXiv preprint arXiv:2306.13649*, 12, 2023.
- [2] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *The twelfth international conference on learning representations*, 2024.
- [3] Howard Chen, Noam Razin, Karthik Narasimhan, and Danqi Chen. Retaining by doing: The role of on-policy data in mitigating forgetting, 2025.
- [4] Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanguan Gu. Self-play fine-tuning converts weak language models to strong language models. *arXiv preprint arXiv:2401.01335*, 2024.
- [5] DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026.
- [6] Jonas Hübötter, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, and Andreas Krause. Reinforcement learning via self-distillation. *arXiv preprint arXiv:2601.20802*, 2026.
- [7] Jeonghye Kim, Xufang Luo, Minbeom Kim, Sangmook Lee, Dohyung Kim, Jiwon Jeon, Dongsheng Li, and Yuqing Yang. Why does self-distillation (sometimes) degrade the reasoning capability of llms? *arXiv preprint arXiv:2603.24472*, 2026.
- [8] Yoon Kim and Alexander M Rush. Sequence-level knowledge distillation. In *Proceedings of the 2016 conference on empirical methods in natural language processing*, pages 1317–1327, 2016.
- [9] Jongwoo Ko, Sungnyun Kim, Tianyi Chen, and Se-Young Yun. Distillm: Towards streamlined distillation for large language models. *arXiv preprint arXiv:2402.03898*, 2024.

²⁴ 【主张I保留】

评：这段把本文机制解释成feedback-enhancement module，而不是替代一切训练目标的统一方案，边界感是清楚的。

- [10] Song Lai, Haohan Zhao, Rong Feng, Changyi Ma, Wenzhuo Liu, Hongbo Zhao, Xi Lin, Dong Yi, Qingfu Zhang, Hongbin Liu, Gaofeng Meng, and Fei Zhu. Reinforcement fine-tuning naturally mitigates forgetting in continual post-training, 2026.
- [11] Gengsheng Li, Tianyu Yang, Junfeng Fang, Mingyang Song, Mao Zheng, Haiyun Guo, Dan Zhang, Jinqiao Wang, and Tat-Seng Chua. Unifying group-relative and self-distillation policy optimization via sample routing. *arXiv preprint arXiv:2604.02288*, 2026.
- [12] Kevin Lu and Thinking Machines Lab. On-policy distillation. *Thinking Machines Lab: Connectionism*, 2025. <https://thinkingmachines.ai/blog/on-policy-distillation>.
- [13] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.
- [14] Emiliano Penaloza, Dheeraj Vattikonda, Nicolas Gontier, Alexandre Lacoste, Laurent Charlin, and Massimo Caccia. Privileged information distillation for language models. *arXiv preprint arXiv:2602.04942*, 2026.
- [15] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.
- [16] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [17] Idan Shenfeld, Mehul Damani, Jonas Hübner, and Pulkit Agrawal. Self-distillation enables continual learning, 2026.
- [18] Idan Shenfeld, Jyothish Pari, and Pulkit Agrawal. RL’s razor: Why online reinforcement learning forgets less, 2025.
- [19] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- [20] Yiyun Sun, Yuhao Cao, Pohao Huang, Haoyue Bai, Hannaneh Hajishirzi, Nouha Dziri, and Dawn Song. RL grokking recipe: How does rl unlock and transfer new algorithms in llms?, sep 2025.
- [21] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.
- [22] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khoshdel, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 13484–13508, 2023.
- [23] Bangjun Xiao, Bingquan Xia, Bo Yang, Bofei Gao, Bowen Shen, Chen Zhang, Chenhong He, Chiheng Lou, Fuli Luo, Gang Wang, et al. Mimo-v2-flash technical report. *arXiv preprint arXiv:2601.02780*, 2026.
- [24] Shusheng Xu, Wei Fu, Jiaxuan Gao, Wenjie Ye, Weilin Liu, Zhiyu Mei, Guangju Wang, Chao Yu, and Yi Wu. Is dpo superior to ppo for llm alignment? a comprehensive study. *arXiv preprint arXiv:2404.10719*, 2024.
- [25] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [26] Chenxu Yang, Chuanyu Qin, Qingyi Si, Minghui Chen, Naibin Gu, Dingyu Yao, Zheng Lin, Weiping Wang, Jiaqi Wang, and Nan Duan. Self-distilled rlvr. *arXiv preprint arXiv:2604.03128*, 2026.

- [27] Tianzhu Ye, Li Dong, Xun Wu, Shaohan Huang, and Furu Wei. On-policy context distillation for language models. *arXiv preprint arXiv:2602.12275*, 2026.
- [28] Ori Yoran, Tomer Wolfson, Ben Bogin, Uri Katz, Daniel Deutch, and Jonathan Berant. Answering questions by meta-reasoning over multiple chains of thought. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5942–5966, 2023.
- [29] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kammanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, et al. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*, 2025.
- [30] Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*, 2026.
- [31] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.
- [32] Pei Zhou, Jay Pujara, Xiang Ren, Xinyun Chen, Heng-Tze Cheng, Quoc V Le, Denny Zhou, Swaroop Mishra, Huaixiu S Zheng, et al. Self-discover: Large language models self-compose reasoning structures. *Advances in Neural Information Processing Systems*, 37:126032–126058, 2024.

A Discussion on the Self-Distillation Objective

As established in Section 2, our unified self-distillation objective minimizes the expected f -divergence between the teacher distribution $\pi_{\theta_{\text{old}}}(\cdot|x, y_{<t}, c)$ and the student policy $\pi_{\theta}(\cdot|x, y_{<t})$. The choice of the generator function f yields qualitatively different optimization landscapes. Below we discuss three canonical instantiations and their practical implications.

Forward Kullback-Leibler (KL) Divergence. Setting $f(\tau) = \tau \log \tau$ recovers the standard distillation objective. Because the loss is an expectation weighted by the student policy π_{θ} yet penalizes deviations through $\tau \log \tau$, the resulting gradient is dominated by regions where the teacher assigns substantial probability mass. This produces *mean-seeking* behavior: the student is penalized heavily whenever it assigns low probability to tokens the teacher considers valid, encouraging it to cover all modes of the teacher distribution.

Reverse KL Divergence. Setting $f(\tau) = -\log \tau$ swaps the directional penalty, yielding *mode-seeking* behavior. Here the student incurs large loss when it generates tokens that the teacher deems highly improbable. In complex reasoning tasks, this property is particularly desirable because it actively suppresses logical hallucinations—tokens that are plausible under the student’s distribution but inconsistent with the teacher’s feedback-informed reasoning.

Jensen-Shannon Divergence (JSD). Setting $f(\tau) = \frac{\tau}{2} \log \tau - \frac{\tau+1}{2} \log(\frac{\tau+1}{2})$ yields a smoothed, symmetric objective bounded between 0 and $\log 2$. By interpolating between the forward and reverse directions, JSD stabilizes training in regimes where the teacher and student distributions are nearly disjoint—precisely the situation that causes extreme gradient explosions under pure forward or reverse KL. This boundedness makes JSD a robust default when the quality gap between teacher and student is large or highly variable across training.

B Success-Rate Rebalancing

In the self-distillation loss, every sample in a rollout batch contributes equally by default. When the batch success rate is very low (or very high), the gradient signal is dominated by the majority outcome, diluting the supervision from the minority. To counteract this imbalance, we weight each sample i by a scalar w_i that depends on the batch success rate $s = |\{i : R(x_i, y_i) \geq \tau\}|/B$:

$$w_i = \begin{cases} (1-s)^{\alpha} & \text{if } R(x_i, y_i) \geq \tau \quad (\text{success}), \\ s^{\beta} & \text{otherwise} \quad (\text{failure}), \end{cases} \quad (2)$$

where $\alpha, \beta > 0$ are hyperparameters that control the strength of rebalancing, and τ is the success reward threshold. The weights are normalized so that their batch mean equals one, $\bar{w} = \frac{1}{B} \sum_i w_i = 1$, ensuring that the effective learning rate is unaffected. The weighted self-distillation loss becomes:

$$\mathcal{L}_{\text{SD}}^w(\theta) = \frac{1}{B} \sum_{i=1}^B w_i \sum_{t=1}^{T_i} \sum_{v \in \mathcal{V}} \pi_{\theta}(v \mid x_i, y_{i, < t}) \cdot f(\tilde{\tau}_v^t). \quad (3)$$

When success is rare ($s \rightarrow 0$), successful samples receive high weights while failed samples receive low weights, amplifying the distillation signal from the few successful demonstrations. Conversely, when success is common, the corrective signal from failures is amplified. In all experiments we set $\alpha = \beta = 1$.

C Training Curves for Main Results

Figure 7 shows the validation training curves of the runs summarized in Table 2. Each panel plots validation accuracy over training steps for one task, comparing the baselines and RESD under the same rank-based checkpoint-selection rule described in Section 4.

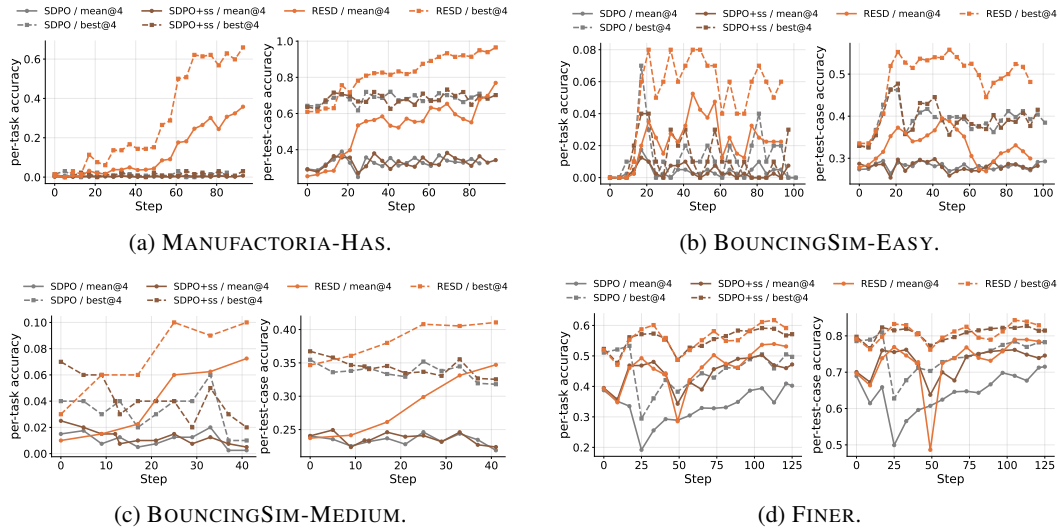


Figure 7: Validation accuracy over training steps for the runs reported in Table 2.

D Playbook Concise Strategy

The CONCISE operation in Section 3.3 maintains the playbook $\mathcal{P}$ as a compact set of reusable lessons over the course of training. We describe two concrete instantiations used in our experiments and the trigger conditions under which they are applied.

Bullet categorization. At the time of each CONCISE call, every entry $p_j \in \mathcal{P}$ is classified based on its tag history. Let h_j and d_j denote the accumulated helpful and harmful counts assigned by the reflector across training steps (Section 3.3). We partition the playbook as

- **Unused:** $h_j + d_j = 0$ (never tagged),
- **Harmful:** $d_j \geq h_j$ and $d_j > 0$ (net harmful),
- **Helpful:** otherwise.

In addition, we record the most recent step τ_j at which each entry was tagged and use $\text{step} - \tau_j$ as a staleness measure.

Concise methods. Given a budget $M_{\max}$ on the playbook size, we consider two removal strategies:

- *Prioritized.* Remove all unused and harmful entries. If $|\mathcal{P}|$ still exceeds $M_{\max}$, randomly drop helpful entries until the cap is met.
- *Staleness.* Remove all harmful entries and all *stale unused* entries (unused for more than one context-update step). If $|\mathcal{P}|$ still exceeds $M_{\max}$, additionally evict the longest-unused remaining entries in order of increasing τ_j until the cap is met.

Both strategies guarantee that persistently harmful entries are removed; they differ in how they treat unused entries. The *staleness* variant is more patient in that unused entries are given one additional step to be tagged before becoming eligible for removal, which helps preserve newly curated lessons that may take a few reflections before receiving a usage signal.

Trigger conditions. CONCISE is invoked under two conditions: (i) a *frequency trigger*, which fires every F context-update steps (default $F = 4$), and (ii) a *limit trigger*, which fires whenever $|\mathcal{P}| > M_{\max}$ between scheduled invocations. In the former case, the playbook is pruned down to its harmful/stale subset without enforcing the budget; in the latter, the budget $M_{\max}$ is enforced.

Default configuration. Unless otherwise noted, we use the *staleness* method with $M_{\max}$ and F set to the values in Appendix C (training details). This is the configuration used for all RESD runs in Table 2.

E Preservation of Instruction Following Capability

Table 5: IFEval accuracy after training on each task.

Checkpoint	Per-Test-Case Acc.		Per-Task Acc.	
	m@4	b@4	m@4	b@4
Base model	87.62	91.33	82.50	87.00
After MANUF.-HAS	87.96	91.33	83.75	87.00
After BSIM-EASY	87.83	92.33	83.25	88.00
After FINER	86.88	90.33	80.50	86.00

A common challenge in continual learning is catastrophic forgetting, where improvements on specific reasoning tasks may degrade the model’s foundational instruction-following capabilities. To verify that RESD avoids this trade-off, we evaluate the models on the IFEval benchmark [31] before and after training. As shown in Table 5, RESD successfully preserves the model’s general capabilities across all evaluation settings. For instance, after training on the MANUFACTORIA-HAS task, the per-task mean@4 accuracy actually increases slightly from 82.50% to 83.75%, while the per-test-case mean@4 accuracy remains stable at around 87–88%.

F Analysis on Training Latency

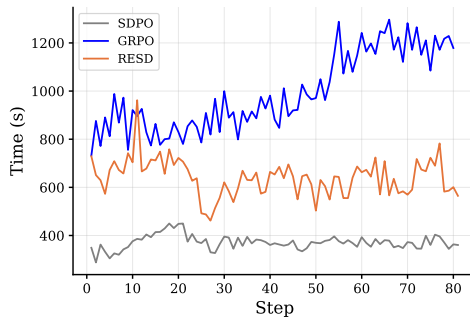


Figure 8: Per-step training latency for BOUNCINGSIM-EASY.

While RESD significantly improves interaction efficiency by requiring only a single rollout per prompt, the generation of retrospective reflections and playbook curation introduces additional inference steps. To understand the latency implications of these components, we analyze the per-step training latency of RESD compared to SDPO and GRPO baselines. As illustrated in Figure 8, standard SDPO exhibits the lowest latency about 400s per-step, because it only computes the teacher distribution and KL loss for a single trajectory. RESD introduces a moderate and stable computational overhead due to the diagnostic inference required for the reflection and curation operations. However, RESD remains substantially faster than GRPO. Because GRPO relies on group-relative optimization, it must repeatedly sample, execute, and evaluate a large batch of trajectories, causing its latency to climb above 1200s per step.

G Implementation Details

Infrastructure. All experiments are conducted on a single node with 8 NVIDIA H200 GPUs. Each experiment completes in approximately one day of wall-clock time. We use vLLM for rollout generation with tensor parallelism across 4 GPUs, and FSDP for distributed training. Rollout generation uses temperature 1.0 with top- p sampling at 0.95. All models have thinking mode enabled.

Training. We train for a single epoch over the training data (online streaming). Each batch undergoes $K = 4$ inner-loop update iterations. The batch size is 32 and we use a single rollout per prompt ($N=1$) for all RESD experiments. We apply a learning rate warmup of 10 steps. For validation, we sample 4 responses per test problem to compute mean@4 and best@4 metrics.

Self-distillation. The self-distillation loss is computed over the top-100 tokens in the student’s vocabulary at each position. Success-rate rebalancing (Appendix B) is applied on BOUNCINGSIM-EASY and FINER.

Context updater (RESD-specific). The playbook is maintained globally (shared across all training examples). Playbook entries are tagged as helpful or harmful during reflection on both failed and successful samples. The solution buffer caches successful trajectories for future teacher prompt construction.

Per-task hyperparameters. Unless otherwise noted, we use EMA update rate 0.0001, lower clip $\epsilon_{\min}=0.2$, and no upper clip. Task-specific deviations are as follows:

- MANUFACTORIA-HAS: reverse KL ($\alpha=1.0$), upper clip $\epsilon_{\max}=5$, max playbook entries $M_{\max}=200$, prioritized concise method, max prompt/response length 49152/20480 tokens.
- BOUNCINGSIM-EASY: JSD ($\alpha=0.5$), max playbook entries $M_{\max}=120$, staleness-based concise method, success-rate reweighting enabled, max prompt/response length 58368/25600 tokens.
- BOUNCINGSIM-MEDIUM: reverse KL ($\alpha=1.0$), EMA update rate 0.01, max playbook entries $M_{\max}=150$, prioritized concise method, max prompt/response length 58368/25600 tokens.
- FINER: JSD ($\alpha=0.5$), max playbook entries $M_{\max}=120$, staleness-based concise method, success-rate reweighting enabled, max prompt/response length 49152/20480 tokens.

GRPO baseline. For GRPO, we use a rollout group size of 8 per prompt with the same learning rate ($1e-6$), batch size (32), and inner-loop steps ($K=4$). GRPO does not use self-distillation, context updating, or EMA.

H Case Study: Full Program Outputs

This appendix provides the complete DSL programs generated at each inner-loop step for the case study in Section 4.6 (“Accept if the tape contains BRBR”). Each listing corresponds to one row of Table 4. We highlight the buggy lines with inline comments.

Step 45 (acc. = 0.00): Parse error from undefined node reference. The program references `state0` in a `PULLER_YG` node, but `state0` is never declared as a node, causing a parse error.

```

START start:
  NEXT node_rb

PULLER_RB node_rb:
  [B] node_b1
  [R] node_rb
  [EMPTY] node_yg

PULLER_YG node_yg:
  [Y] node_rb
  [G] node_rb
  [EMPTY] state0      # BUG: 'state0' is never declared

PULLER_RB node_b1:
  [R] node_b1r1
  [B] node_b1
  [EMPTY] node_yg

...

END end

```

Step 46 (acc. = 0.84): Correct pattern matching logic, but rejects non-R/B characters. The state machine correctly implements substring matching transitions for B and R. However, all [EMPTY] branches route to NONE, which causes immediate rejection when a non-R/B character (Y or G) is encountered—e.g., input GGYYYBBRBR is rejected at state s0 because the leading G triggers [EMPTY] NONE.

```

START start:
  NEXT s0

PULLER_RB s0:
  [B] s1
  [R] s0
  [EMPTY] NONE      # BUG: rejects Y/G characters

PULLER_RB s1:
  [R] s2
  [B] s1
  [EMPTY] NONE      # BUG: same issue

PULLER_RB s2:
  [B] s3
  [R] s0
  [EMPTY] NONE      # BUG: same issue

PULLER_RB s3:
  [R] end
  [B] s1
  [EMPTY] NONE      # BUG: same issue

END end

```

Step 47 (acc. = 0.96): Overcorrection introduces an infinite loop. The model adds a PULLER_YG node to handle non-R/B characters, fixing the step 46 bug. However, when the tape becomes empty at a non-terminal state, PULLER_RB routes to state0_yg, whose [EMPTY] routes to NONE—but if there are remaining characters that alternate between the two node types, they bounce back and forth infinitely.

```

START start:
  NEXT state0

PULLER_RB state0:
  [B] state1
  [R] state0
  [EMPTY] state0_yg      # Routes non-R/B to PULLER_YG

PULLER_RB state1:
  [R] state2
  [B] state0
  [EMPTY] state0_yg

PULLER_RB state2:
  [B] state3
  [R] state0
  [EMPTY] state0_yg

PULLER_RB state3:
  [R] end
  [B] state0
  [EMPTY] state0_yg

PULLER_YG state0_yg:
  [Y] state0
  [G] state0
  [EMPTY] NONE           # BUG: causes infinite loop when tape
                          # is empty between state0 and state0_yg

END end

```

Step 48 (acc. = 1.00): Correct solution. The final program correctly handles all cases: non-pattern symbols are skipped via dedicated PULLER_YG nodes, and empty-tape transitions at non-terminal states correctly reject. A cached successful solution from a prior rollout provided the complete exemplar that guided this output.

```

START start:
  NEXT s0

PULLER_RB s0:
  [B] s1
  [R] s0
  [EMPTY] s0_yg

PULLER_YG s0_yg:
  [Y] s0
  [G] s0
  [EMPTY] NONE

PULLER_RB s1:
  [R] s2
  [B] s1
  [EMPTY] s1_yg

PULLER_YG s1_yg:
  [Y] s1
  [G] s1
  [EMPTY] NONE

PULLER_RB s2:
  [B] s3
  [R] s0
  [EMPTY] s2_yg

PULLER_YG s2_yg:
  [Y] s2
  [G] s2
  [EMPTY] NONE

PULLER_RB s3:
  [R] end
  [B] s1
  [EMPTY] s3_yg

PULLER_YG s3_yg:
  [Y] s3
  [G] s3
  [EMPTY] NONE

END end

```